Longest Increasing Subsequence

최장 증가 부분 수열

김학재

2026.03.25

목차

1. 문제 설명
2. 최장 증가 부분 수열
3. 완전탐색
4. DP
5. 이분탐색 LIS

문제 설명

수열 = a1, a2, a3 …. aN 이 주어졌을 때

가장 긴 증가하는 부분수열의 원소 개수는 무엇인가?

5 4 2 3 7 6 8

2378
2368 -> 4개

가장 긴 증가하는 부분수열

수열 { 3 5 2 7 }

부분수열 ex) {3}, {5 2}, {2 7}, {3 5 7}

3 <- 증가하는 부분수열
5 2 <- X
2 7 <- 증가하는 부분수열
3 5 7 <- 가장 긴 증가하는 부분수열

완전탐색

수열 { 3 5 2 7 }

부분수열 총 개수 : 2^n

3 , 5 , 2 , 7

35 , 32 , 37 , 52 , 57 , 27

352 , 357 , 327 , 527

3527 , ∅

2^4 : 16

2^{30} : 약 10억

시간복잡도 : 2^n

DP (동적 계획법)

LIS문제를 풀기전에

ex) 피보나치 수열

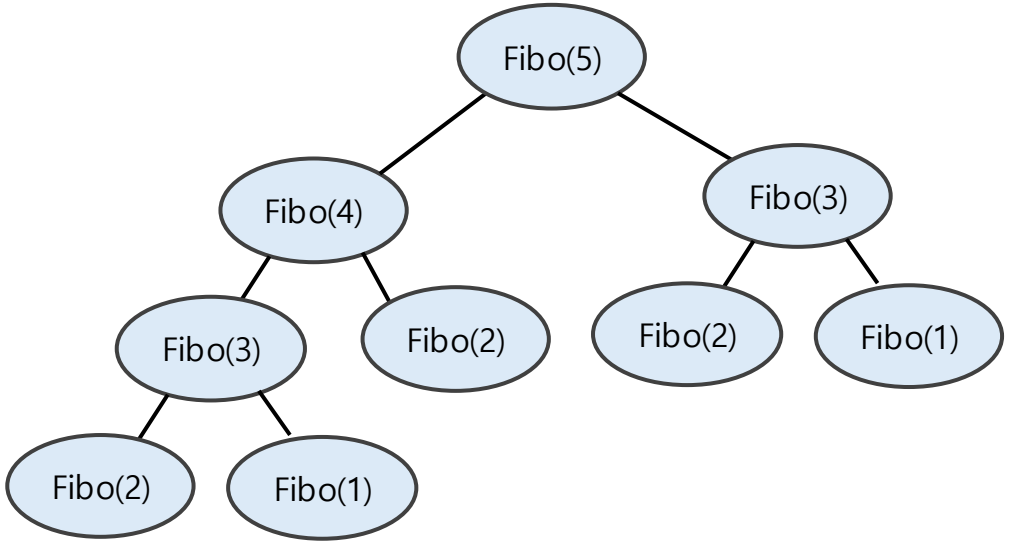
1, 1, 2, 3, 5, 8 …

idx 1 2 3 4 5 6

재귀 :

```
fibo(int N) {
    if (N <= 2) return 1;
    return fibo(N-1) + fibo(N-2);
}
```

시간복잡도 : 2^n



DP (동적 계획법)

과거에 구한 값을 활용하는 알고리즘

arr

[0]	[1]	[2]	[3]	[4]
1	**1**	**2**		

arr[N] = arr[N-1] + arr[N-2]

DP :

```
arr[0] = 1;
arr[1] = 1;
for (int i=2; i<=n; i++) {
    arr[i] = arr[i-1] + arr[i-2];
}
```

피보나치 수열 dp 활용 시 시간복잡도 : N

DP를 활용한 최장 증가 부분수열

	idx	0	1	2	3	4
주어진 수열	arr	**3**	**5**	**2**	**7**	**4**
가장 긴 수열 길이	dp	**1**	**2**	**1**	**3**	**2**

1개 짜리 부분수열도 길이 1로 취급한다

dp[i]의 값을 채우려면 arr[0]~arr[i-1]을 확인한다

dp[i] : i번째 원소를 마지막으로하는 LIS의 최대길이

```
for (int i=0; i<n; i++) {
    for (int j=0; j<i; j++) {
        if (arr[i] > arr[j])
            dp[i] = max(dp[i], dp[j] + 1);
    }
}
```

시간복잡도 : n^2

이분탐색 LIS

	idx	0	1	2	3	4
주어진 수열	arr	**3**	**5**	**2**	**7**	**4**

배열	L	**2**	**4**	**7**		

방법 : 수열을 차례로 받아서

1. L 안에 자신보다 큰 수가 있으면 그 수를 대체한다.
2. 없으면 배열 L 뒤에 추가한다.

결과 : 최장 증가 부분수열의 길이 : 배열 L의 길이

하지만 2,4,7이 실제 최장 증가 부분수열은 아니다

이분탐색 LIS

	idx	0	1	2	3	4
주어진 수열	arr	**3**	**5**	**2**	**7**	**4**
가장 긴 수열 길이 (자신이 최후미 일 때)	dp	1	2	1	3	2
배열	L	**2**	**4**	**7**		
	길이	1	2	3	4	5

L은 각 길이별로 가능한 가장 작은 값을 저장하는 것이다.

L[0] = 길이 1짜리가 가능한 가장 작은 값
L[1] = 길이 2짜리가 가능한 가장 작은 최후미 값
L[k] = k+1인 증가부분수열 중 가장 작은 최후미 값.
-> L의 size가 최장증가 부분수열의 정답이 된다.

이분탐색을 활용한 LIS

```
int binary_lower(int arr[], int size, int target) {
    int left = 0;
    int right = size;

    while (left < right) {
        int mid = (left + right) / 2;
        if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid;
    }
    return right;
}
```

※STL존재 lower_bound(arr, arr+n, target)

정렬된 배열에서 target이상이 처음 등장하는 위치 반환

이분탐색 lower_bound

시간복잡도 : log N

```
int len = 0; // 현재 L 길이
for (int i = 0; i < n; i++) {
    int pos = binary_lower(L, len, arr[i]);

    if (pos == len) {
        L[len] = arr[i]; // 뒤에 추가
        len++;
    }
    else {
        L[pos] = arr[i];   // 값 교체
    }
}
```

N번 반복

-> 총 시간복잡도 : N log N

결론 (최장 증가 부분수열의 길이 구하기)

	N : 1000	N : 1000000
완전탐색 : 2^n	약 10^{301}	-
DP : n^2	1,000,000	10^{12}
이분탐색 : n log n	10,000	$2\ X10^7$

LIS 문제를 풀기에 좋은 것은 이분탐색을 활용한 방법이다.
N의 크기가 크지 않다면 구현이 쉬운 것은 DP 방법이 쉽다.

참고

문제를 풀어보고싶다면

백준

- 11053번. 가장 긴 증가하는 부분 수열
- 11055번. 가장 큰 증가하는 부분 수열
- 11722번. 가장 긴 감소하는 부분 수열
- 12015번. 가장 긴 증가하는 부분 수열 2
- 12738번. 가장 긴 증가하는 부분 수열 3
- 14002번. 가장 긴 증가하는 부분 수열 4
- 14003번. 가장 긴 증가하는 부분 수열 5

가장 긴 증가하는 부분수열의 길이에 대해서만 다뤘는데,
그 수열의 원소가 무엇인지까지 다루는 문제도 존재한다

감사합니다